第6章 MCS-51微型计算机系统原理及应用

6.3 MCS-51汇编程序设计

§6.3.1 MCS-51汇编伪指令

汇编语言语句一般分为指令性语句和指示性语句。

- 指令性语句是采用助记符构成的汇编语言语句，在汇编时能产生与之对应的机器码，供CPU识别，从而执行相应的功能。
- 指示性语句又称伪指令语句。伪指令不是真正的指令，虽然与指令的形式类似，但在汇编时不产生供CPU执行的任何机器代码。
- 伪指令起到指示说明的作用，在机器汇编时供汇编程序识别和执行的命令，用于在汇编过程中对数据的存储环境或汇编实施一定的控制。

1、ORG 起始汇编伪指令

[标号：]　ORG 16位的地址或标号

汇编过程中，机器检测到该语句时，便确认了汇编的起始地址，然后把ORG伪指令下一条指令的首字节机器码存入16位的地址或标号所指示的存储单元内，其他的后续指令字节或数据连续依次存入后面的存储单元中。

例：　　ORG 1000H
　START：MOV A，#45H；

结果：（1000H）=74H，（1001H）=45H

2、END 汇编结束伪指令

END

汇编程序检测到该语句时，认为汇编语言源程序已经结束，END后面的指令语句将不被汇编成机器码。一个程序中包括主程序和若干个子程序，只能有一个END语句，它在所有程序单位的最末尾。

3、EQU 赋值伪指令(等值)

字符名　EQU　数据或汇编符

其功能是将一个数或者特定的汇编符号赋予规定的符号名称。

例如：

```
INP   EQU   33H
ADD   A，INP
```

这里将INP等值为33H.

“字符名”必须先赋值后使用，因此EQU伪指令通常放在源程序的开头。

例：

```
ORG 1000H
KK    EQU R0
H1    EQU 20H
DISP  EQU 0300H
      MOV R1, H1;
      MOV A, KK;
      ...
      ACALL DISP
      ...
      END
```

例：

```
ORG    0000H
MOV A, CC;
   ...
LCALL DISP
   ...
CC    DATA   20H
DISP  XDATA  0B00H
END
```

4、DATA 数据地址赋值伪指令

字符名 DATA 表达式

它用来给左边的“字符名”赋值。表达式可以是数据、地址，也可以是包含被定义的“字符名”在内的表达式，但不能是汇编符号，如R0-R7等。

(1)EQU伪指令定义的符号必须先定义后使用，而DATA伪指令的符号可以先使用后定义；

(2)用EQU伪指令可以把一个汇编符号赋给一个字符名称，而DATA伪指令则不能。

(3)DATA伪指令可将一个表达式的值赋给一个字符名称，所定义的字符名称也可以出现在表达式中，而用EQU定义的字符则不能这样使用。

(4)DATA伪指令常在程序中用来定义数据地址。

5、DB 定义字节伪指令

[标号]：DB 项或项表

从规定的地址单元开始，定义若干个8位内存单元的内容。在程序存储器的某一部分存入一组规定好的8位二进制数，或者是将一个数据表格存入程序存储器。这个伪指令在汇编后，将影响程序存储器的内容。

例如：　TAB1：DB　3FH，55，‘8’，‘C’

　　TAB2：DB　10100B

若TAB1的对应地址为2000H，则以上伪指令经汇编以后，将对2000H开始的若干内存单元赋值：

（2000H）=3FH　　（2001H）=37H

（2002H）=38H　　（2003H）=43H

（2004H）=14H

6.DW 定义字伪指令

[标号]：DW 项或项表

DW与DB的功能类似，区别在于DB定义的是字节，DW定义的是字。DW经常被用来定义16位的地址，16位地址的存放顺序是高8位在前，低8位在后。

例如：

```
ORG 1000H
DW  3964H，06H，20
```

汇编以后结果为：

（1000H）=39H （1001H）=64H （1002H）=00H

（1003H）=6H （1004H）=00H （1005H）=14H

7. DS预留存储空间伪指令

[标号:] DS 表达式

该伪指令的功能是从标号指定的单元开始，保留若干字节的内存空间以备源程序使用。存储空间内预留的存储单元数由表达式的值决定。

例如:

```
ORG     1000H
DS      10
DB      30H, 8FH
```

从1000H开始,预留10个字节的内存单元,然后从1010H开始,按照下一条DB指令赋值,即(100AH)=30H,(100BH)=8FH。保留的存储空间将由程序的其它部分决定它们的用处。

8.BIT 位地址赋值伪指令

字符名 BIT 位地址

其功能是将位地址赋予所规定的字符名称。

例：X1 BIT 30H

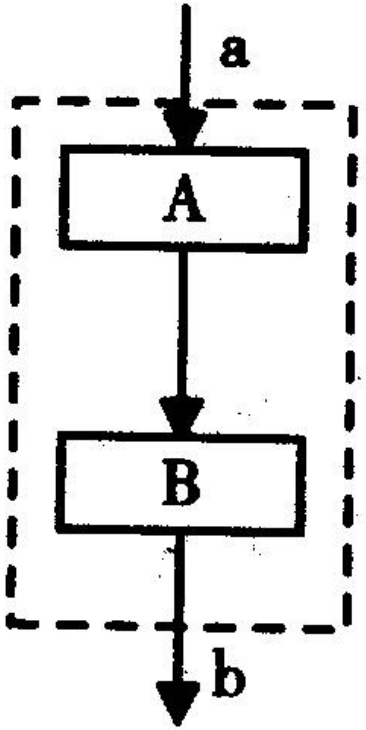
RECORD BIT P2.2

把两位位地址分别赋给两个变量X1和RECORD，在编程中它们可当作位地址来使用。但不是所有的MCS-51汇编程序都有这条伪指令。当不具备BIT命令时，可以使用EQU命令来定义位地址变量。

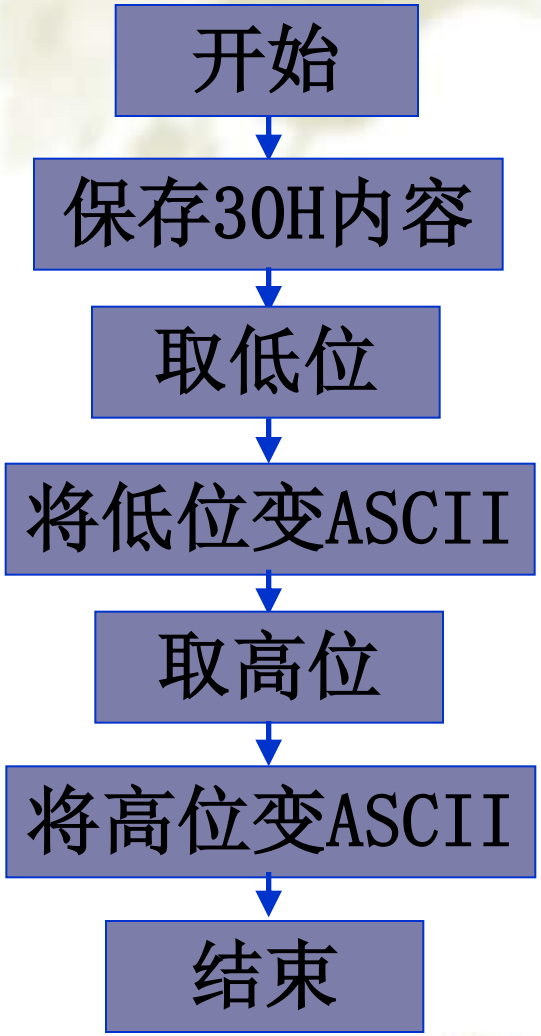
§6.3.2 MCS-51汇编程序设计

程序的基本算法结构有3种:顺序结构、分支（选择）结构和循环结构。

1、顺序结构：



例：将30H内的两位压缩BCD码拆开，并转换成相应的ASCII码，存入两个RAM单元31H和32H中（低位存31H）




```
ORG 1000H
MOV R0，#30H;
MOV SP，#50H;
MOV A，@R0;
PUSH ACC;
ANL A，#0FH;
ORL A，#30H;
INC R0;
MOV @R0，A;
POP ACC;
ANL A，#0F0H;
SWAP A;
ORL A，#30H;
INC R0;
MOV @R0，A;
END
```

例：将40H单元中8位无符号二进制数，转换成3位BCD码，并存在BAI(百)，SHI(十)，GE(个)三个单元中。

```
ORG 2000H
BAI DATA 30H
SHI DATA 31H
GE  DATA 32H
MOV  A, 40H
MOV  B, #100
DIV  AB ;A/100,商存A,余数存B
MOV  BAI,A;存百位数
MOV  A, B
MOV  B, #10
DIV  AB ;A/10,商存A,余数存B
MOV  SHI, A;存十位数
MOV  GE, B
SJMP $
END
```

2、分支程序设计

根据不同的条件，执行不同的程序段。直接用来判断分支条件的指令有JZ、JNZ、CJNE、JC、JNC、JB、JNB等。

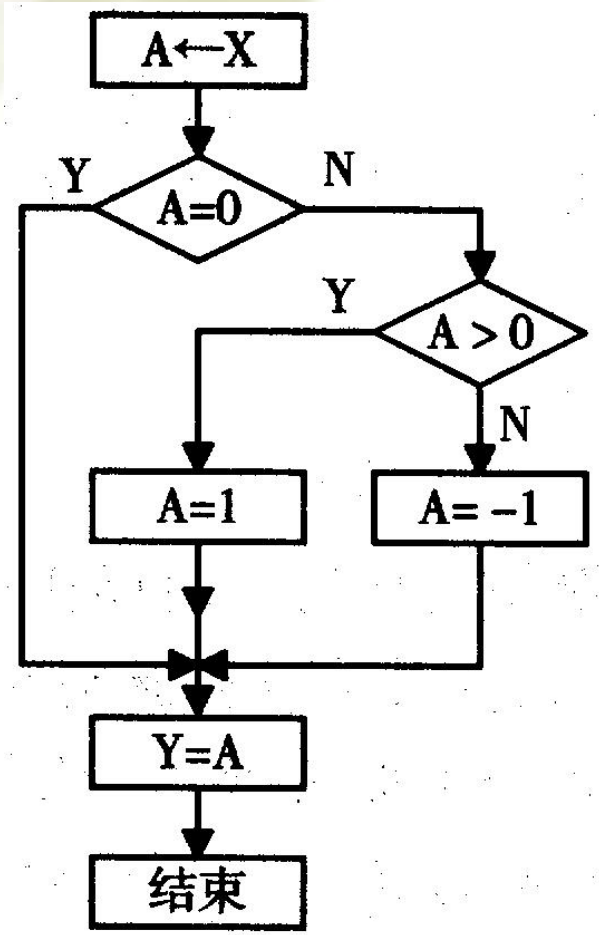
例：在内部RAM50H单元中有一数，若其最高位为1，则将此数各位变反，反之不变。

```
        ORG  1000H
        MOV  A，50H;
        JNB  ACC.7,HERE;
        CPL  A;
        MOV  50H,A;
HERE:SJMP  HERE;
        END
```

```
        ORG  1000H
        MOV  A，50H;
        ANL  A，#80H;
        CJNE  A,#80H,HERE;
        MOV  A，50H;
        CPL  A;
        MOV  50H,A;
HERE:SJMP  HERE;
        END
```

例：设变量X存放于R2，函数值Y存放于R3，按下式的要求给Y赋值。

$$Y=\begin{cases}1, & X>0\\ 0, & X=0\\ -1, & X<0\end{cases}$$




```
      ORG 0500H
      MOV A, R2;
      CJNE A, #00H, L1;
      MOV A, #00H;
      SJMP L3;
L1:JB ACC.7,L2
      MOV A,#01H;
      SJMP L3;
L2:MOV A,#0FFH;
L3:MOV R3,A
      SJMP L3;
      END
```

$$Y=\begin{cases}1, & X>0\\ 0, & X=0\\ -1, & X<0\end{cases}$$

```
        ORG 0500H
        MOV A, R2;
        JZ L2;
        JB ACC.7,L1
        MOV R3, #01H;
        SJMP L3;
L1:MOV R3,#0FFH;
        SJMP L3;
L2:MOV R3, #00H
L3:SJMP L3;
        END
```

3、循环程序设计

循环程序一般由四部分组成：

1） 循环初始化

循环初始化程序段位于循环程序的开头，用于完成循环前的准备工作。例如：给循环体中计数器和各工作寄存器设置初值。

2） 循环处理

这部分程序位于循环体内，是循环程序重复执行的部分。

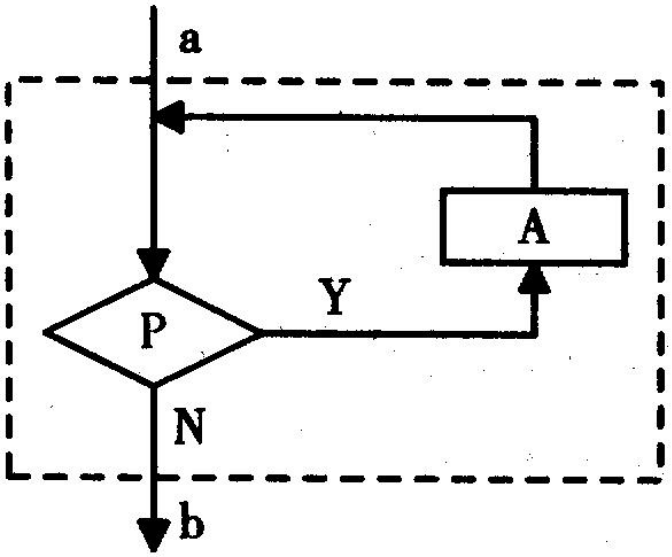
3） 循环控制

循环控制在循环体内，由修改循环控制变量和条件转移语句等组成，用于控制循环执行次数。

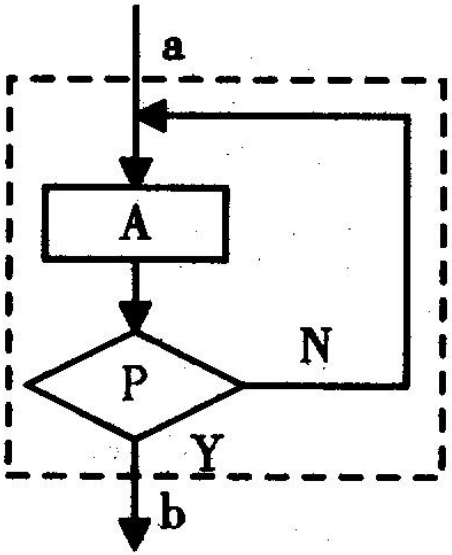
4） 循环结束

用于存放执行循环程序所得结果以及恢复各单元结果

循环结构：它在一定的条件下，反复执行某一部分的操作。循环结构又分为当型（While）循环结构和直到型（Until）循环结构两种方式。 当型循环是先判断条件，条件成立则执行循环体A;而直到型循环则是先执行循环体A一次,再判断条件,条件不成立再执行循环体A。循环结构的两种形式可以互相转换。



(a) 当型循环



(b) 直到型循环

例：已知以30H和50H为首地址的存储区中分别有5个字节无符号被减数和减数（低位在前，高位在后），编写一多字节减法程序，把差放入以30H为起始地址的存储单元。

```
        ORG 4000H
        MOV R0，#30H；
        MOV R1，#50H；
        MOV R2，#05H；
        CLR C；
LOOP：  MOV  A，@R0；
        SUBB A，@R1；
        MOV @R0，A；
        INC  R0；
        INC  R1；
        DJNZ R2，LOOP；
        SJMP $；
        END
```

例：把内部RAM中起始地址为BLK1的数据块传送到外部RAM中以BLK2为起始地址的区域，直到遇到“#”字符的ASCII码为止。数据块长度放在R1中。

```
        ORG 2000H
BLK1    EQU 30H
BLK2    EQU 1000H
        MOV R0，#BLK1；
        MOV DPTR，#BLK2；
XH：    CLR C；
        MOV A，@R0；
        CJNE A，#23H，L1；
        SJMP JS；
L1：    MOVX @DPTR，A；
        INC R0；
        INC DPTR；
        DJNZ R1，XH；
JS：    SJMP JS；
        END
```

例：已知内部RAM31H为起始地址的数据块内数据是无符号数，块长在30H内，编程求数据块中最大值并存入29H单元中。

```
        ORG 0300H
        MOV R0，#31H；
        MOV 29H，#00H；
LOOP：  MOV A，@R0；
        CJNE A，29H，L1；
  L1：  JC NEXT；
        MOV 29H，A；
NEXT：  INC R0；
        DJNZ 30H，LOOP；
        SJMP $
        END
```

CJNE只用于无符号整数的比较。

4、查表程序设计

两条查表指令：

MOVC A，@A+DPTR；

MOVC A，@A+PC；

例：已知R0中有一个十六进制数（0-F中的一个），编程把它转换成相应的ASCII码并送入R0。

```
ORG 0100H
MOV DPTR，#TAB;
MOV A，R0;
MOVC A，@A+DPTR;
MOV R0，A;
SJMP $
```

```
TAB：DB ‘0’,
‘1’ DB‘2’5,’,3’,’4’
‘6’ ,DB‘7’‘A’,’8’,’9’
‘B’ END‘C’,’D’,’E’,’F’
```

5、子程序设计

子程序是指完成某一确定任务并能被其他程序反复调用的程序段。子程序在结构上应具有通用性和独立性

例：在R2中有两位16进制数，试将它们分别转换成ASCII，并存入30H和31H单元中。

```
ORG 4000H
MOV SP，#50H;
MOV A，R2;
ACALL HASC;
MOV 30H，A;
MOV A，R2;
SWAP A;
ACALL HASC;
MOV 31H，A;
SJMP $;
```

```
HASC：MOV DPTR，#TAB;
      ANL A，#0FH;
      MOVC A，@A+DPTR;
      RET

TAB：DB  '01234567'
     DB  '89ABCDEF'

      END
```

例：编制$C=A^2+B^2$ 程序，设A和B均小于10的整数，ABC放在内部RAM的XA、XB、XC三个单元中。

```
ORG 2000H
XA DATA 40H
XB DATA 41H
XC DATA 42H
MOV A，XA；
ACALL SQR；
MOV XC，A；
MOV A，XB；
ACALL SQR；
ADD A，XC；
MOV XC，A；
SJMP $；
```

```
SQR；MOV B，A；
     MUL AB；
     RET
     END
```

例：已知内RAM32H单元开始有一无符号数据块，块长在30H单元中，求数据块中各数累加和并存入31H单元中（设所加之和不超过8位数）。

```
        ORG 0200H
        MOV R2，30H；
        MOV R1，#32H；
        CLR A；
LOOP：  ADD A，@R1；
        INC R1；
        DJNZ R2，LOOP；
        MOV 31H，A；
        SJMP $
        END
```

```
        ORG 0200H
        MOV R2，30H；
        MOV R1，#32H；
        CLR A；
        INC R2；
        SJMP CHECK；
LOOP：  ADD A，@R1；
        INC R1；
CHECK：DJNZ R2，LOOP
        MOV 31H，A；
        SJMP $
        END
```

例：已知以30H和50H为首地址的存储区中分别有5个字节无符号被减数和减数（低位在前，高位在后），编写一多字节减法程序，把差放入以30H为起始地址的存储单元。

```
        ORG 4000H
        MOV R0，#30H;
        MOV R1，#50H;
        MOV R2，#05H;
        CLR C;

LOOP：  MOV  A，@R0;
        SUBB A，@R1;
        MOV @R0，A;
        INC  R0;
        INC  R1;
        DJNZ R2，LOOP;
        SJMP  $；
        END
```

例：内存中以STRING开始的区域有若干个字符和数字，一般称为一个字符串，最末一个字符为“$”，试统计这些字符的数目，结果存入NUM单元。

符号“$”的ASCII码是24H。

```
ORG      0300H
NUM      DATA    20H
STRING   DATA    21H
         CLR A                      ；A清零
         MOV R0，#STRING            ；首地址送R0
LOP：    CJNE @R0，#24H，LOP2 ；与$不等转移
         SJMP  LOP3                 ；找到$，结束循环
LOP2：   INC A                      ；计数器加1
         INC R0                     ；修改地址指针
         SJMP LOP                   ；循环
LOP3：   INC A                      ；再计这个$字符
         MOV NUM，A                 ；存结果
         END
```

例:设在内部RAM的BLOCK单元开始处有长度为LEN个的无符号数据块，试编一个求和程序，并将和存入内部RAM的SUM单元（设和不超过8位）。

```
        ORG   0300H
BLOCK   EQU   20H
LEN     EQU   30H
SUM     EQU   40H
START:  CLR   A            ; 清累加器A
        MOV   R2, #LEN     ; 数据块长度送R2
        MOV   R1, #BLOCK;  数据块首址送R1

LOOP:   ADD   A, @R1       ; 循环加法
        INC   R1;  修改地址指针
        DJNZ  R2, LOOP     ; 修改计数器并判断
        MOV   SUM, A;  存和
        END
```

例：已知内部RAM31H为起始地址的数据块内数据是无符号数，块长在30H内，编程求数据块中最大值并存入29H单元中。

```
        ORG 0300H

        MOV R0，#31H；
        MOV 29H，#00H；

LOOP：MOV A，@R0；
      CJNE A，29H，L1；

   L1：JC NEXT；
       MOV 29H，A；

NEXT：INC R0；
      DJNZ 30H，LOOP；

        SJMP $

      END
```

CJNE只用于无符号整数的比较。

例：把内部RAM中起始地址为BLK1的数据块传送到外部RAM中以BLK2为起始地址的区域，直到遇到“#”字符的ASCII码为止。数据块长度放在R1中。

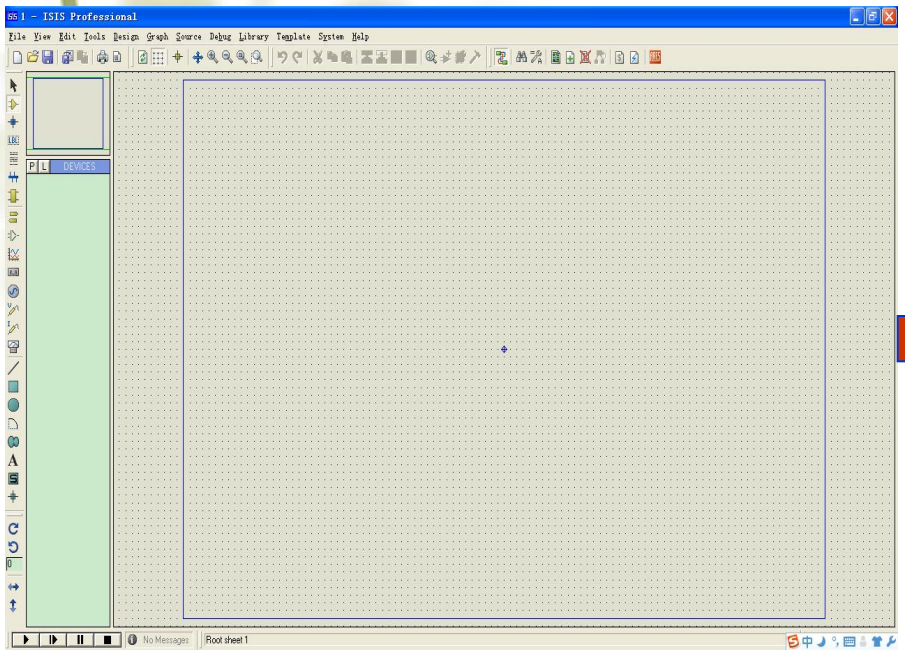
```
        ORG 2000H
BLK1 EQU 30H
BLK2 EQU 1000H
     MOV R0，#BLK1；
     MOV DPTR，#BLK2；
 XH：CLR C；
     MOV A，@R0；
     CJNE A，#23H，L1；
     SJMP JS；
L1：MOVX @DPTR，A；
    INC R0；
    INC DPTR；
    DJNZ R1，XH；
JS：SJMP JS；
    END
```

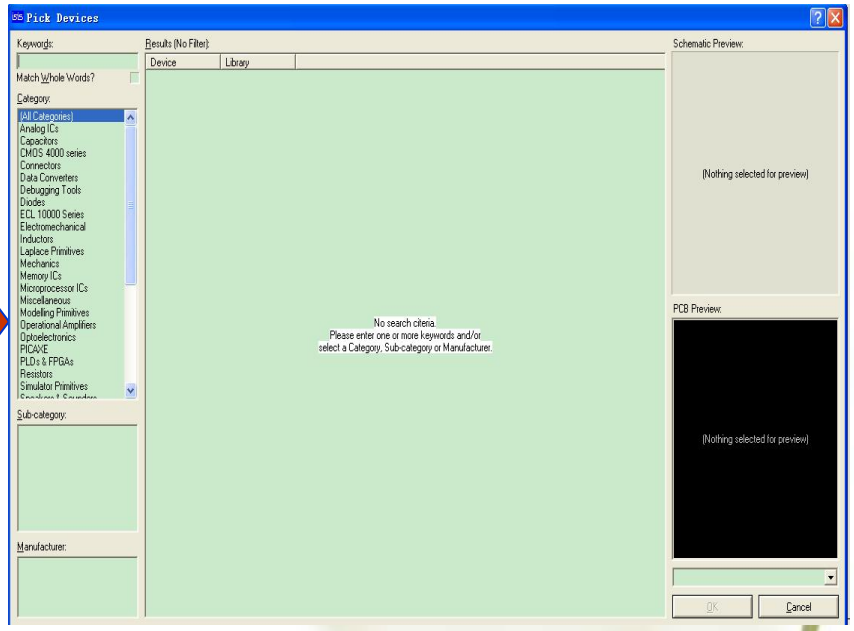
§6.3.3 MCS-51仿真软件PROTEUS

1、Proteus集成环境 ISIS

Proteus软件包提供一种界面友好的人机交互式集成环境 ISIS，其设计功能强大，使用方便。

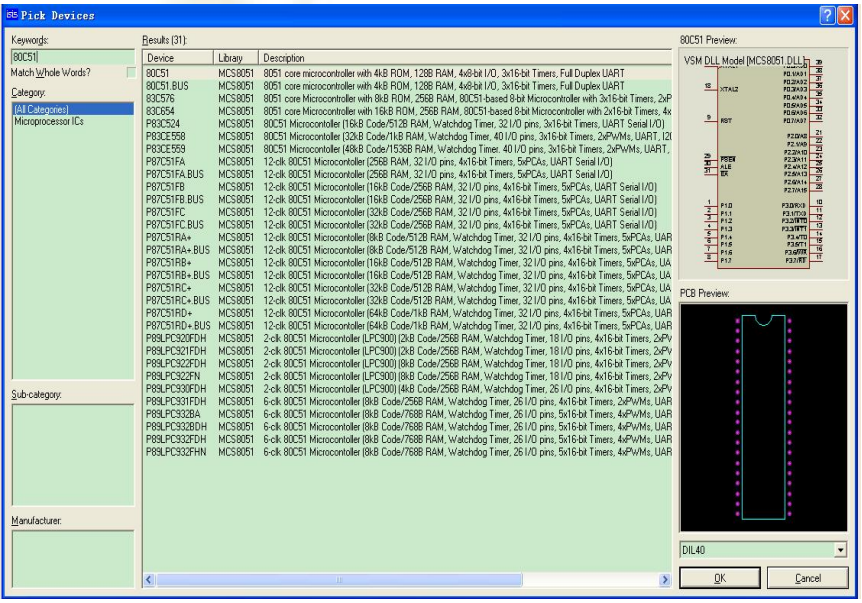
ISIS在Windows环境下运行，启动后弹出如图7-18所示界面。

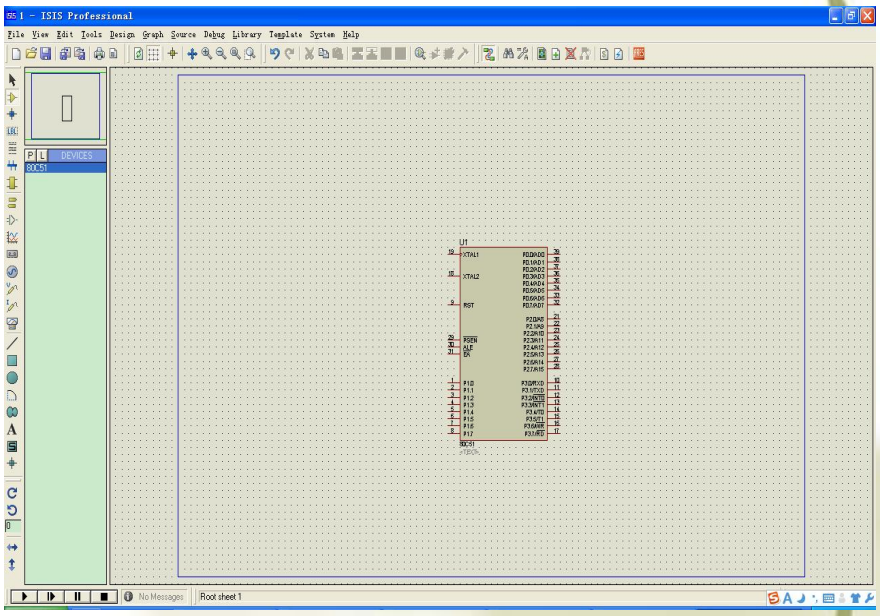




2、绘制原理图，首先应选取元器件，Proteus ISIS库中提供了大量元器件原理图符号，利用Proteus ISIS的搜索功能很方便地查找需要的元器件。

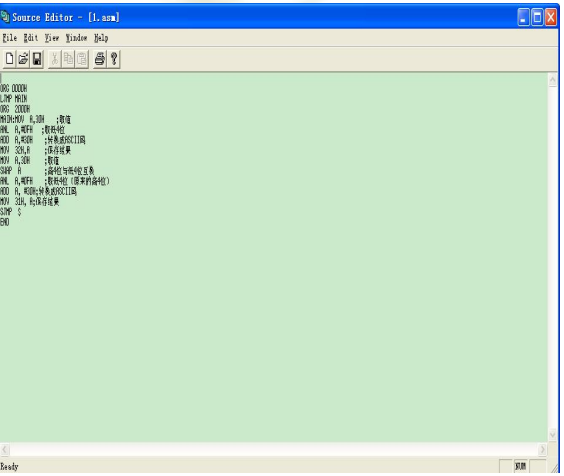
如80C51，则可得如图6-20所示的搜索结果界面。点击OK按钮即可将80 C51单片机放置在绘图区域，如图6-21所示。

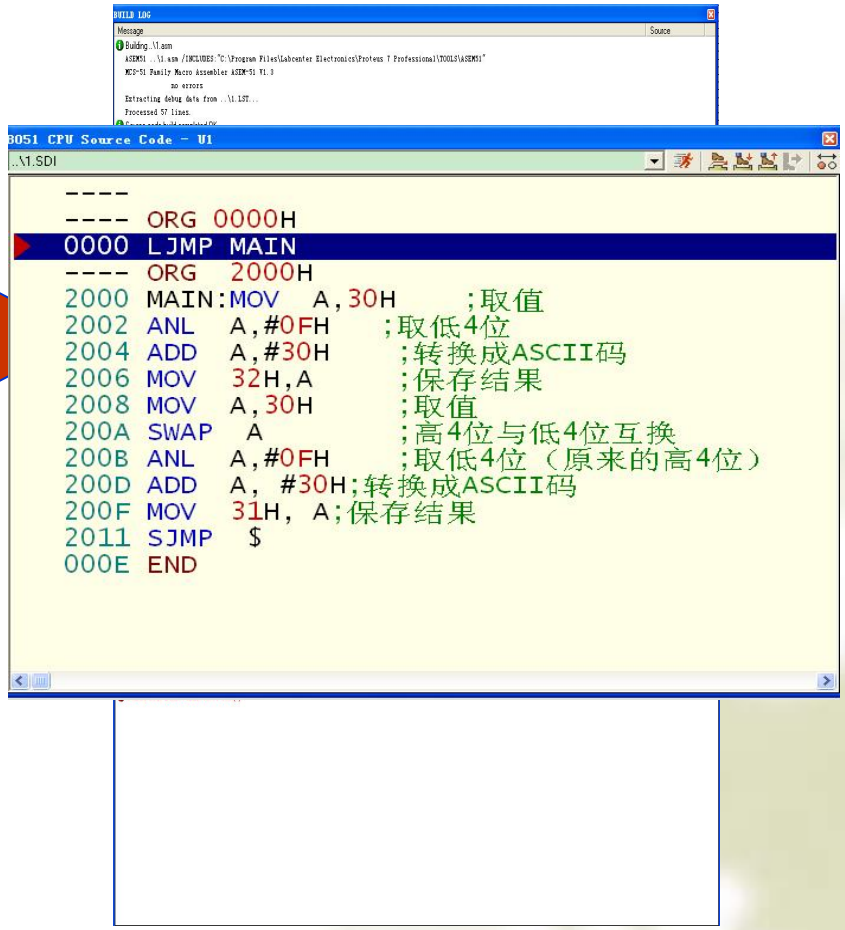




3、创建源代码仿真文件

Proteus虚拟仿真系统将源代码的编辑与编译整合在同一设计环境中





8051 CPU Internal (IDATA) Memory - U1

00	00	02	00	00	00	00	00	00	.....
08	C5	01	69	01	F3	01	00	00	..i.....
10	00	00	00	00	00	00	00	00	.....
18	00	00	00	00	00	00	00	00	.....
20	00	00	00	00	00	00	00	00	.....
28	00	00	00	00	00	00	00	00	.....
30	00	00	00	00	00	00	00	00	.....
38	00	00	00	00	00	00	00	00	.....
40	00	00	00	00	00	00	00	00	.....
48	00	00	00	00	00	00	00	00	.....
50	00	00	00	00	00	00	00	00	.....
58	00	00	00	00	00	00	00	00	.....
60	00	00	00	00	00	00	00	00	.....
68	00	00	00	00	00	00	00	00	.....
70	00	00	00	00	00	00	00	00	.....
78	00	00	00	00	00	00	00	00	.....

8051 CPU SFR Memory - U1

80	FF	00	B9	FF	00	00	00	00	.....
88	10	01	F2	00	FF	00	02	00	.....
90	FF	00	00	00	00	00	00	00	.....
98	00	00	00	00	00	00	00	00	.....
A0	FF	00	00	00	00	00	00	00	.....
A8	00	00	00	00	00	00	00	00	.....
B0	FF	00	00	00	00	00	00	00	.....
B8	00	00	00	00	00	00	00	00	.....
C0	00	00	00	00	00	00	00	00	.....
C8	00	00	00	00	00	00	00	00	.....
D0	00	00	00	00	00	00	00	00	.....
D8	00	00	00	00	00	00	00	00	.....
E0	FF	00	00	00	00	00	00	00	.....
E8	00	00	00	00	00	00	00	00	.....
F0	00	00	00	00	00	00	00	00	.....
F8	00	00	00	00	00	00	00	00	.....

8051 CPU Registers - U1

PC	INSTRUCTION
2011	SJMP 2011

ACC	B	DPTR	SP	CA-rsO-P
30	00	0000	07	00000000

R0	R1	R2	R3	R4	R5	R6	R7
00	00	00	00	00	00	00	00

P0	P1	P2	P3	SCON	SBUF
FF	FF	FF	FF	00	00

TMR0	TMR1	TMOD	TCON	PCON
0000	0000	00	00	00

IE	IP	TMR2	TCON	RCAP
00	00	0000	00	0000